

NLP

nltk spacy tensorflow

textblob Gensim

✓ Text Preprocessing

Tokenization Lemmetization Stemming Stopwords Normalization Reg Ex

Feature Extraction

Bag of Words word Embeddings

NLP Models

Rule Based - RegEx ML - Logistic Regression, SVM, NB DL - RNN, Transformers

Applications

Sentiment Analysis Classification Text Q&A Chatbots

```
import nltk
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.corpus import stopwords
import gensim
```

```
!pip uninstall -y numpy scipy gensim && pip install gensim
```

```
Found existing installation: numpy 1.26.4
Uninstalling numpy-1.26.4:
  Successfully uninstalled numpy-1.26.4
Found existing installation: scipy 1.13.1
Uninstalling scipy-1.13.1:
  Successfully uninstalled scipy-1.13.1
Found existing installation: gensim 4.3.3
Uninstalling gensim-4.3.3:
  Successfully uninstalled gensim-4.3.3
Collecting gensim
  Using cached gensim-4.3.3-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (8.1 kB)
Collecting numpy<2.0,>=1.18.5 (from gensim)
  Using cached numpy-1.26.4-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (61 kB)
Collecting scipy<1.14.0,>=1.7.0 (from gensim)
  Using cached scipy-1.13.1-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (60 kB)
Requirement already satisfied: smart-open>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.3.0.post1)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart-open>=1.8.1->gensim) (1.17.3)
Using cached gensim-4.3.3-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (26.6 MB)
Using cached numpy-1.26.4-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (18.0 MB)
Using cached scipy-1.13.1-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (38.2 MB)
Installing collected packages: numpy, scipy, gensim
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is temporary and will be removed in a future version. To work with this version, please use --no-deps.
opencv-contrib-python 4.12.0.88 requires numpy<2.3.0,>=2; python_version >= "3.9", but you have numpy 1.26.4 which is incompatible.
opencv-python 4.12.0.88 requires numpy<2.3.0,>=2; python_version >= "3.9", but you have numpy 1.26.4 which is incompatible.
tsfresh 0.21.1 requires scipy>=1.14.0; python_version >= "3.10", but you have scipy 1.13.1 which is incompatible.
thinc 8.3.6 requires numpy<3.0.0,>=2.0.0, but you have numpy 1.26.4 which is incompatible.
opencv-python-headless 4.12.0.88 requires numpy<2.3.0,>=2; python_version >= "3.9", but you have numpy 1.26.4 which is incompatible.
Successfully installed gensim-4.3.3 numpy-1.26.4 scipy-1.13.1
```

```
nltk.download('punkt')
nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt_tab.zip.
True
```

```
text = "NLP is Powerful and Useful"
```

```
print(word_tokenize(text))
```

```
['NLP', 'is', 'Powerful', 'and', 'Useful']
```

Stemming - prefixes/ suffixes Studing - studi Goodies - Good better - better

Lemme - Meaning Real word Studying - Study better - good

```
from nltk.stem import PorterStemmer, WordNetLemmatizer
import spacy
from textblob import Word
```

Double-click (or enter) to edit

```
nlp = spacy.load("en_core_web_sm")
doc = nlp(text)
```

doc

NLP is Powerful and Useful

```
for tokens in doc :
    print(tokens.text, tokens.lemma_ , tokens.pos_)
```

```
NLP NLP PROP
is be AUX
Powerful powerful ADJ
and and CCONJ
Useful useful ADJ
```

```
words = ["Studying", " Running" , "Better" , "Dogs", "Rainfall"]
```

```
nltk.download('wordnet')
```

```
[nltk_data] Downloading package wordnet to /root/nltk_data...
True
```

```
stemmer = PorterStemmer()
print([stemmer.stem(word) for word in words])
```

```
['studi', ' run', 'better', 'dog', 'rainfal']
```

```
lemmetizer = WordNetLemmatizer()
print([lemmetizer.lemmatize(word, pos='v') for word in words])
```

```
['Studying', ' Running', 'Better', 'Dogs', 'Rainfall']
```

```
from textblob import Word, TextBlob
```

```
txt = "This World has a lot of Faces "
```

```
parse_data = TextBlob(text).words
print(parse_data )
```

```
['NLP', 'is', 'Powerful', 'and', 'Useful']
```

```
[(word , word.lemmatize()) for i , word in enumerate(parse_data) if word != parse_data[i].lemmatize() ]
```

```
[]
```

```
import pandas as pd
```

```
text = " Google is looking at buying U.K. startup for $1 billion and Running it in Circles"
doc = nlp(text)
pd.DataFrame(
    [(word.text, word.lemma_ , word.pos_ , word.tag_ , word.dep_ ,
      word.shape_ , word.is_alpha , word.is_stop) for word in doc]
)
```

	0	1	2	3	4	5	6	7	
			SPACE	_SP	dep		False	False	
0									
1	Google	Google	PROPN	NNP	nsubj	Xxxxx	True	False	
2	is	be	AUX	VBZ	aux	xx	True	True	
3	looking	look	VERB	VBG	ROOT	xxxx	True	False	
4	at	at	ADP	IN	prep	xx	True	True	
5	buying	buy	VERB	VBG	pcomp	xxxx	True	False	
6	U.K.	U.K.	PROPN	NNP	nsubj	X.X.	False	False	
7	startup	startup	VERB	VBD	ccomp	xxxx	True	False	
8	for	for	ADP	IN	prep	xxx	True	True	
9	\$	\$	SYM	\$	quantmod	\$	False	False	
NER 10	1	1	NUM	CD	compound	d	False	False	
11	billion	billion	NUM	CD	pobj	xxxx	True	False	
for entity in nlp(text).ents:									
print(entity.text, entity.label_)									
12	Running	run	VERB	VBD	advcl	xxxx	True	False	
13	Google	ORG							
14	U.K.	GPE	it	it	PRON	PRP	dobj	xx	True
15	\$1 billion	MONEY							
16	Circles	GPE	in	ADP	IN	prep	xx	True	True
17									
18	Circles	circle	NOUN	NNS	pobj	Xxxxx	True	False	
#Preprocessing									
# Stemming									
# Lemmetizatin									
# POS									
# Stop Words									
#NLTK									
#SPACY									
#TensorFlow									

from string import digits

Regex

/^[a-z][0-9_]

\d - digits [a-z] {m} * + \s (...) (a(bc))

match - search - findall - sub -

```
string = "Tiger is our National Animal, Today 16-09-2025"
```

```
import re
```

```
pattern = r'\d{2}-\d{2}-\d{4}'
result = re.search(pattern, string).group(0)
print(result)
```

```
16-09-2025
```

```
#Tensorflow
```

```
from tensorflow.keras.preprocessing.text import Tokenizer
```

```
sentences = ['i love my dog', 'I, Love my car' , 'You Love my House!', 'Do you think my hat is Amazing?']
```

```
tokenizer = Tokenizer(num_words=100, oov_token="<oov>")
tokenizer.fit_on_texts(sentences)
word_index = tokenizer.word_index
print(word_index)
```

```
{' < o o v > ': 1, ' m y ': 2, ' l o v e ': 3, ' i ': 4, ' y o u ': 5, ' d o g ': 6, ' c a r ': 7, ' h o u s e ': 8, ' d o ': 9, ' t h i n k ': 10, ' h a t ': 11, ' i s ': 12,
```

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
```

```
sequences = tokenizer.texts_to_sequences(sentences)
print(sequences)
```

```
[[4, 3, 2, 6], [4, 3, 2, 7], [5, 3, 2, 8], [9, 5, 10, 2, 11, 12, 13]]
```

```
padded = pad_sequences(sequences, maxlen=5)
print(padded)
```

```
[[ 0 4 3 2 6]
 [ 0 4 3 2 7]
 [ 0 5 3 2 8]
 [10 2 11 12 13]]
```

```
pad_seq = pad_sequences(sequences, maxlen=10, padding='post')
print(pad_seq)
```

```
[[ 4 3 2 6 0 0 0 0 0 0]
 [ 4 3 2 7 0 0 0 0 0 0]
 [ 5 3 2 8 0 0 0 0 0 0]
 [ 9 5 10 2 11 12 13 0 0 0]]
```

```
#BERT - Classifier
#Tensorflow Graph
#Preprocessing - Model Inference
```

```
# Embeddings
```

```
#Word2vec
#tfidf
#onhotencoding
#bagofwords
#glove
# Text Embeddings - All the AI Models
# Google - Universal Sentence Encoder
```

```
doc
```

```
Google is looking at buying U.K. startup for $1 billion and Running it in Circles
```

```
S1 = list(doc.sents)[0]
```

```
S1.vector
```

```
array([-1.9160900e-01, -1.1894695e-01, 3.7544793e-01, 2.6300594e-01,
       3.1219110e-01, -7.8881137e-02, 3.3776170e-01, 7.8865543e-02,
       2.5634015e-01, 1.2801822e-01, -5.5747068e-01, 1.5661141e-01,
       -2.1048994e-01, 7.6473594e-02, 4.2466548e-01, 2.6419932e-01,
       -1.5225144e-01, 1.5614825e-01, -1.6382284e-01, -7.2856724e-02,
       1.9938225e-01, 4.5370671e-01, 2.3171844e-02, -5.7431541e-02,
       -6.8184314e-03, -4.4831339e-01, 3.4479913e-01, 3.3525351e-01,
       -1.8404146e-01, -6.2539212e-02, -2.0065327e-01, 1.0279219e-01,
       7.1025640e-02, -8.5635208e-02, -2.6608148e-01, -1.8824139e-01,
       1.9815688e-01, 3.7590098e-01, 3.7693295e-01, -6.2169630e-02,
       -1.3695383e-01, -7.5462952e-02, -2.0907408e-01, 3.0972103e-02,
       4.0649107e-01, 1.1514690e-01, 4.5434918e-02, -5.2195340e-02,
       3.2583666e-01, -7.8066856e-02, 1.2174730e-01, -2.2456348e-01,
       -1.5814960e-01, -6.1185867e-01, -4.0403107e-01, -3.0137104e-01,
       1.4524259e-02, -3.5981333e-01, 3.0322650e-01, -8.5447900e-02,
       -1.9581879e-04, -7.2765462e-02, -1.1808951e-02, -6.0714014e-02,
       -3.9363432e-01, 2.0453353e-01, 1.2669905e-01, -2.0943178e-01,
       -1.2143701e-01, 1.5500647e-02, -1.5822372e-01, -2.5033987e-01,
       4.7310602e-02, 5.9004370e-02, -6.7680344e-02, -5.2005309e-01,
       -1.9044413e-01, -8.5624643e-02, 7.7224262e-02, -1.6815010e-01,
       -8.1958085e-02, 3.0669644e-01, -7.4891910e-02, -2.6918358e-01,
       1.2111727e-01, 1.4599977e-01, 4.1368082e-01, -1.8613598e-01,
       -4.3594527e-01, 6.4942145e-01, 5.6351550e-02, 6.4040720e-02,
       5.4777227e-03, 2.4356696e-01, -8.4590375e-02, -6.0518742e-02],
      dtype=float32)
```

```
doc2 = nlp("Open AI is already having U.K Startups bid at $2 Billion")
```

```
print(doc2.similarity(doc))
```

```
0.6497670528624054
```

```
/tmp/ipython-input-3171705594.py:1: UserWarning: [W007] The model you're using has no word vectors loaded, so the result of the
print(doc2.similarity(doc))
```

```
# Pipelines
```

```
# Input - Entity Recognition
```

```
nlp.add_pipe('sentencizer')
```

```
<spacy.pipeline.sentencizer.Sentencizer at 0x792f77022d10>
```

```
from sklearn.feature_extraction.text import CountVectorizer
vectorizer = CountVectorizer()
```

```
print(vectorizer.fit_transform(sentences).todense())
print(vectorizer.vocabulary_)
```

```
[[0 0 0 1 0 0 0 1 1 0 0]
 [0 1 0 0 0 0 0 1 1 0 0]
 [0 0 0 0 0 1 0 1 1 0 1]
 [1 0 1 0 1 0 1 0 1 1 1]]
{'love': 7, 'my': 8, 'dog': 3, 'car': 1, 'you': 10, 'house': 5, 'do': 2, 'think': 9, 'hat': 4, 'is': 6, 'amazing': 0}
```

```
sentences
```

```
['i love my dog',
 'I, Love my car',
 'You Love my House!',
 'Do you think my hat is Amazing?']
```

```
# TFIDF
```

```
from sklearn.feature_extraction.text import TfidfVectorizer
vectorizer = TfidfVectorizer(stop_words='english')
tfidf = vectorizer.fit_transform(sentences)
print(tfidf.shape)
print(vectorizer.get_feature_names_out())
```

```
(4, 7)
['amazing' 'car' 'dog' 'hat' 'house' 'love' 'think']
```

```
print(tfidf.toarray())
```

```
[[0.         0.         0.84292635 0.         0.         0.53802897
  0.         ]
 [0.         0.84292635 0.         0.         0.         0.53802897
  0.         ]
 [0.         0.         0.         0.         0.84292635 0.53802897
  0.         ]
 [0.57735027 0.         0.         0.57735027 0.         0.
  0.57735027]]
```

```
!pip install gensim
```

```
Requirement already satisfied: gensim in /usr/local/lib/python3.12/dist-packages (4.3.3)
Requirement already satisfied: numpy<2.0,>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.26.4)
Requirement already satisfied: scipy<1.14.0,>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.13.1)
Requirement already satisfied: smart-open>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.3.0.post1)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart-open>=1.8.1->gensim) (1.17.3)
```

```
import gensim
from gensim.models import Word2Vec
```

```
model = Word2Vec(sentences)
```

```
WARNING:gensim.models.word2vec:Each 'sentences' item should be a list of words (usually unicode strings). First item here is ir
```

```
print(model)
```

```
Word2Vec<vocab=4, vector_size=100, alpha=0.025>
```

```
words = list(model.wv.key_to_index)
print(words)
```

```
[' ', 'o', 'y', 'm']
```